6. Classical Forecasting Models — LEAR

From Baselines to Real Models

Chapter 5 established the baselines — the similar-day naive and simple autoregressive models. These are surprisingly hard to beat, but they have fundamental limitations: they use only the target variable's own past values and calendar indicators. They ignore weather, demand, cross-hour relationships, and every other signal that might improve predictions.

This chapter introduces the first "real" forecasting model: **LEAR** (LASSO Estimated AutoRegressive). LEAR is a linear model, but one armed with a powerful trick — **automatic feature selection** via LASSO regularisation. It consistently performs near the top of academic electricity price forecasting competitions while remaining simple, fast, and interpretable. Understanding LEAR in depth is essential because it serves as the benchmark that all subsequent machine learning models must beat.

Linear Regression: The Foundation

What Is Linear Regression?

Before understanding LEAR, we need to understand the building block it rests on: **linear regression**.

> **Linear regression:** A statistical method that models the relationship between a target variable (the thing you want to predict) and one or more features (the things you use to predict it) as a weighted sum. The model assumes the target is approximately equal to a constant plus a weighted combination of the features. The weights (called **coefficients** or **parameters**) are estimated from historical data by minimising the sum of squared prediction errors.

In its simplest form, with one feature:

$y* = \beta_0 + \beta_1 \cdot x_1$

where: - y* is the predicted value (the model's output) - x_1 is the feature (the model's input) - β_0 is the **intercept** (the predicted value when $x_1 = 0$) - β_1 is the **coefficient** (how much y* changes when x_1 increases by one unit)

With multiple features:

$y* = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + ... + \beta_p \cdot x_p$

> **Coefficient (weight, parameter):** A number that quantifies the relationship between a feature and the target. A coefficient of $\beta_1 = 0.6$ on the lag-48 price feature means: "For each $1 increase in yesterday's same-hour price, today's predicted price increases by $0.60, holding all other features constant." Coefficients are estimated from data, not chosen by the modeller.

> **Intercept (bias term):** The constant term β_0 in a linear model. It represents the predicted value when all features are zero. In electricity price forecasting, the intercept captures the baseline price level that is not explained by any feature.

Ordinary Least Squares (OLS)

The standard method for estimating the coefficients is **Ordinary Least Squares (OLS)**: choose the coefficients that minimise the sum of squared errors between the predictions and the actual values.

> **Ordinary Least Squares (OLS):** The method of estimating linear regression coefficients by minimising the sum of squared residuals: $\Sigma(y_i - y*_i)^2$. This is equivalent to finding the coefficients that make the model's predictions as close as possible to the actual values, where "close" is measured by squared distance. OLS has a closed-form solution (the "normal equations"), so the coefficients can be computed exactly without iteration.

Minimise: $(1/N) \times \Sigma_{i=1}^{N} (y_i - y*_i)^2$

> **Residual:** The difference between an actual value and the model's prediction: $e_i = y_i - y*_i$. A positive residual means the model under-predicted (actual was higher than predicted); a negative residual means the model over-predicted. The goal of regression is to make residuals as small as possible.

OLS works beautifully when you have a moderate number of well-chosen features. But it breaks down when the feature set is large, redundant, or noisy — precisely the situation in electricity price forecasting.

The Problem with Too Many Features

Consider a LEAR-style model with features for each hour h:

- 3 lagged prices at hour h (lag-1, lag-2, lag-7 days)
- 23 cross-hour prices from yesterday (all other hours)
- 6 day-of-week indicators

- 11 month indicators
- Demand forecast
- Wind speed, solar CSI, temperature

That is already 45+ features per hour. OLS will happily fit all 45 coefficients — but many of these features are irrelevant for any given hour (solar features at 3am, for example), and the resulting model **overfits**: it learns noise in the training data rather than genuine patterns.

> **Overfitting:** When a model learns the noise (random variation) in the training data rather than the underlying signal (true patterns). An overfit model performs excellently on the training data but poorly on new, unseen data. Overfitting is more likely when the model has many parameters relative to the amount of training data — exactly the situation when using a large feature set with OLS.

> **Real-world example — overfitting in action:** Suppose we include 100 features in an OLS model trained on 365 days of data. Many features will be correlated with price purely by coincidence — perhaps rainfall in Darwin happens to correlate with SA1 prices during the training period due to a shared seasonal pattern. The OLS model assigns a non-zero coefficient to this spurious feature. On new data, the spurious correlation disappears, and the model's predictions are degraded by the nonsensical rainfall coefficient. LASSO would have set this coefficient to zero, correctly discarding the irrelevant feature.

Regularisation: Controlling Model Complexity

The Bias-Variance Tradeoff

Every forecasting model faces a fundamental tension between two sources of error:

> **Bias:** Error from overly simplistic assumptions in the model. A model with high bias "underfits" — it fails to capture genuine patterns in the data. For example, a model with only an intercept (no features) has high bias because it predicts the same constant for every hour, ignoring the duck curve.

> **Variance:** Error from the model's sensitivity to the specific training data. A model with high variance "overfits" — its predictions change substantially if you swap the training data for a different sample from the same period. An OLS model with 100 features has high variance because the coefficient estimates are unstable when features are correlated or the sample is not large enough.

> **Bias-variance tradeoff:** The principle that reducing bias (making the model more flexible) tends to increase variance (making it more sensitive to training data), and vice versa. The optimal model balances these two sources of error. Regularisation is the primary tool for navigating this tradeoff — it intentionally adds bias (by constraining coefficients) to reduce variance (by preventing overfitting).

In time-series forecasting, the tradeoff has a **temporal dimension**:

- **More training data** (longer window) reduces variance but increases bias if the market has changed.
- **More features** can reduce bias (capturing more signals) but increases variance (more parameters to estimate).
- **Regularisation** deliberately increases bias (by shrinking or eliminating coefficients) to reduce variance.

The goal is to find the sweet spot — a model complex enough to capture the real signals but simple enough to generalise to new data.

Ridge Regression (L2 Penalty)

The first regularisation approach, **Ridge regression**, adds a penalty on the *squared magnitude* of the coefficients:

> **Ridge regression (L2 regularisation):** A modification of OLS that adds a penalty term proportional to the sum of squared coefficients: $\lambda \times \Sigma \beta_j^2$. This penalty discourages large coefficients, pulling them toward zero. The result is a model with smaller, more stable coefficients that generalises better to new data. However, Ridge never sets any coefficient exactly to zero — it shrinks all coefficients but keeps them all in the model.

Minimise: $(1/N) \times \Sigma (y_i - y\hat{}_i)^2 + \lambda \times \Sigma_j \beta_j^2$

The parameter λ (lambda) controls how aggressively the coefficients are shrunk: - $\lambda = 0$: No regularisation. Identical to OLS. - $\lambda \to \infty$: All coefficients shrink to zero. The model predicts a constant (the mean).

Ridge works well when all features contribute some information and you want to dampen their influence — but it does not perform **feature selection**. Even truly irrelevant features retain small but non-zero coefficients, adding noise to predictions.

LASSO Regression (L1 Penalty)

LASSO replaces the squared penalty with an **absolute value** penalty:

> **LASSO (Least Absolute Shrinkage and Selection Operator):** A regularised linear regression method that adds a penalty proportional to the sum of the *absolute values* of the coefficients: $\lambda \times \Sigma |\beta_j|$. The L1 penalty has a remarkable mathematical property: it drives some coefficients exactly to zero, effectively removing those features from the model. LASSO simultaneously estimates coefficients and performs feature selection — it decides which features matter and which do not.

Minimise: $(1/N) \times \Sigma_{i=1}^{N} (y_i - y\hat{}_i)^2 + \lambda \times \Sigma_{j=1}^{p} |\beta_j|$

L1 penalty (L1 norm): The sum of the absolute values of the coefficients: $\Sigma |\beta_j|$. The "L1" notation comes from the mathematical concept of the L^1 norm of a vector. The L1 penalty creates a "diamond-shaped" constraint region in coefficient space, which has corners on the coordinate axes. The optimal solution often falls at these corners, where one or more coefficients are exactly zero. This geometric property is why L1 produces sparse solutions while L2 (circle-shaped constraint) does not.

Sparsity: A property of a model where most coefficients are exactly zero — only a few features have non-zero weights. A sparse model is easier to interpret (you can see which features matter), faster to evaluate (only non-zero features need to be computed), and often more accurate on new data (irrelevant features are excluded rather than contributing noise).

Why L1 Creates Sparsity and L2 Does Not

The difference between LASSO (L1) and Ridge (L2) comes down to geometry. Visualise the problem in two dimensions (two features with coefficients β_1 and β_2):

- The OLS objective (sum of squared errors) creates elliptical contours in coefficient space. The OLS solution is at the centre of the ellipses.
- **Ridge** constrains the solution to lie within a circle ($\beta_1^2 + \beta_2^2 \leq t$). The constrained optimum is where the smallest ellipse touches the circle — which is generally at a point where both β_1 and β_2 are non-zero (the circle has no corners).
- **LASSO** constrains the solution to lie within a diamond ($|\beta_1| + |\beta_2| \leq t$). The constrained optimum is where the smallest ellipse touches the diamond — which often occurs at a **corner** of the diamond, where one coefficient is exactly zero.

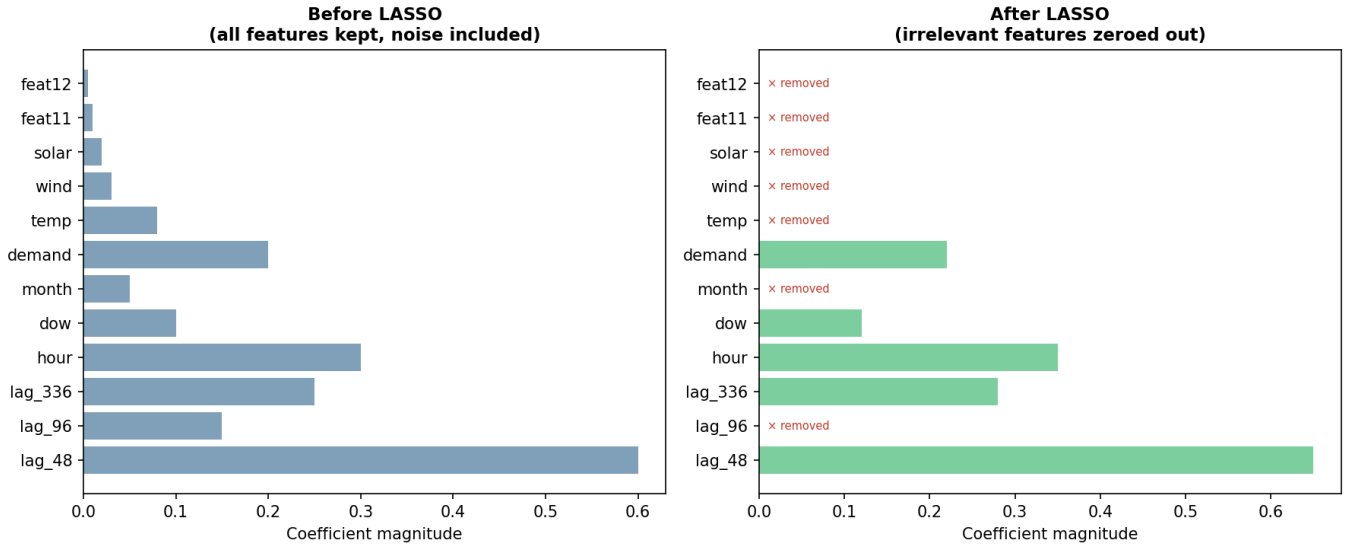

Figure 6.1 — The geometry of LASSO (L1) vs Ridge (L2) regularisation. The diamond-shaped L1 constraint has corners on the axes, where one coefficient is exactly zero. The OLS contour ellipses are more likely to touch the diamond at a corner than to touch the circle at an axis intercept. This is why LASSO produces sparse solutions — it sets irrelevant coefficients to exactly zero.

The geometric intuition: LASSO drives coefficients to exactly zero because its constraint region has sharp corners on the coordinate axes. When the OLS solution touches a corner, one or more coefficients become exactly zero. Ridge's circular constraint has no corners, so the solution generically has all coefficients non-zero. This seemingly small geometric difference — diamond vs. circle — has profound practical consequences: LASSO performs automatic feature selection while Ridge does not.

The Regularisation Parameter λ

The parameter λ controls the tradeoff between fitting the data (small residuals) and keeping the model simple (small/zero coefficients):

λ value	Effect	Risk
λ = 0	No regularisation. All features used. Equivalent to OLS.	Overfitting
Small λ	Gentle regularisation. Most features retained, slightly shrunk.	Mild overfitting
Moderate λ	Moderate regularisation. Many irrelevant features eliminated.	Good balance
Large λ	Heavy regularisation. Most features eliminated.	Underfitting
$\lambda \to \infty$	All coefficients zero. Model predicts a constant.	Maximum underfitting

Regularisation parameter (λ, also called α in scikit-learn): The hyperparameter that controls the strength of the regularisation penalty. A larger λ penalises large coefficients more aggressively, producing a simpler (sparser) model. The optimal λ is not known in advance — it must be selected by evaluating model performance across a range of candidate values, typically using cross-validation.

Hyperparameter: A setting that controls the behaviour of the learning algorithm itself, as opposed to the model's parameters (coefficients), which are learned from data. The regularisation strength λ is a hyperparameter — it is set before training begins and determines *how* the coefficients are learned, not *what* they are. Other hyperparameters include the number of cross-validation folds, the maximum number of iterations, and the choice of solver.

LassoCV: Automatic λ Selection

Cross-Validation for λ

How do you choose the right λ? Trying it on the training data does not work — $\lambda = 0$ (no regularisation) always gives the lowest training error, but overfits. You need to evaluate on data the model has not seen.

Cross-validation (CV): A procedure for estimating how well a model generalises to unseen data, by repeatedly splitting the training data into a "fit" subset and a "validation" subset. The model is trained on the fit subset and evaluated on the validation subset. The average validation performance across all splits provides a robust estimate of generalisation error. Cross-validation is used to select hyperparameters (like λ) that cannot be estimated from the training data directly.

LassoCV automates this process:

1. Define a grid of candidate λ values (typically 100 values, logarithmically spaced from large to small).
2. For each λ, perform K-fold cross-validation: split the training data into K subsets, train on K−1 subsets, validate on the remaining one, and rotate.
3. Select the λ that minimises the average validation error.
4. Refit the model on the full training data using the selected λ.

LassoCV: An implementation of LASSO regression (available in scikit-learn) that automatically selects the optimal regularisation parameter λ using cross-validation. The user does not need to specify λ — LassoCV tries many values and picks the best one. This makes LEAR essentially **parameter-free** from the user's perspective: there are no tuning knobs to adjust.

Regularisation path: The sequence of LASSO solutions as λ varies from large (all coefficients zero) to small (many non-zero coefficients). As λ decreases, features "enter" the model one by one — the most important features enter first (their coefficients become non-zero at the largest λ values), and the least important enter last. The regularisation path provides a natural ranking of feature importance.

Cross-Validation for Time Series

Standard K-fold cross-validation randomly shuffles the data into K groups. This is **invalid for time series** because it violates temporal ordering — the model might train on Monday's data and validate on the preceding Friday's data, creating look-ahead leakage.

Time-series cross-validation (expanding window CV): A variant of cross-validation designed for temporally ordered data. Instead of random splits, the data is divided into K consecutive blocks. In fold k, the model trains on blocks 1 through k and validates on block k+1. This preserves temporal ordering — the model always trains on past data and validates on future data, mimicking the real forecasting scenario.

Blocked cross-validation: An alternative to expanding-window CV where the K folds are contiguous time blocks but the training set does not necessarily include all preceding blocks. This can be useful when you want to avoid the expanding-window assumption (that all past data is always relevant). Both approaches are valid; the key requirement is that validation data always comes *after* training data in time.

Never use random-shuffle CV on time series: Standard K-fold cross-validation (as implemented by default in scikit-learn's `cross_val_score`) randomly assigns observations to folds, destroying temporal ordering. A model trained on November data and validated on October data has seen the future. Always use `TimeSeriesSplit` or a custom blocked-CV scheme for time-series problems.

The LEAR Model

Background and Motivation

The **LASSO Estimated AutoRegressive (LEAR)** model was introduced by Lago, Marcjasz, De Schutter, and Weron in their 2021 paper "Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark." It has become the standard benchmark for day-ahead electricity price forecasting worldwide.

LEAR's central insight is that electricity prices have a **strong hourly structure**: the statistical relationships between features and prices differ substantially between different times of day. At 3am, the price is driven by baseload demand and coal generation; solar features are irrelevant (the sun is down). At 1pm, the price is dominated by the solar surplus and the midday demand dip; last night's prices are less informative. At 6pm, the evening ramp drives the price, and the transition from solar to gas peakers is the key mechanism.

Rather than forcing a single model to learn these hour-specific patterns (which would require complex interaction terms), LEAR fits **separate models for each hour of the day**.

The LEAR Formulation

For each hour h in {0, 1, ..., 23} (or each half-hour h in {0, 1, ..., 47} for half-hourly data), LEAR fits a LASSO regression:

$y*_{d,h} = \beta_0 h + \Sigma_j \beta\_jh \cdot x_j,d + \varepsilon_{d,h}$

where: - *y*{d,h} is the predicted price for day d, hour h* - *$\beta_0 h$ is the intercept for hour h* - *β_jh is the coefficient for feature j at hour h — note the superscript h, indicating that each hour has its own set of coefficients* - *x_j,d is the value of feature j on day d* - ε is the error term

The feature vector x typically includes:

> **LEAR (LASSO Estimated AutoRegressive):** A day-ahead electricity price forecasting model that fits separate LASSO regressions for each hour (or half-hour) of the day. Each model receives a large set of candidate features — lagged prices (including cross-hour lags), calendar indicators, demand, and weather — and LASSO's L1 penalty automatically selects the relevant features for that specific hour. The "autoregressive" part of the name refers to the inclusion of lagged prices as features. LEAR is fast, interpretable, and consistently competitive with far more complex models.

1. Same-hour lagged prices: - $y_{d-1,h}$: Yesterday's price at hour h (lag-1 day) - $y_{d-2,h}$: Two days ago at hour h (lag-2 day) - $y_{d-7,h}$: Last week's price at hour h (lag-7 day)

2. Cross-hour lagged prices: - $y_{d-1,h-1}$, $y_{d-1,h+1}$: Neighbouring hours yesterday - Potentially all 24 (or 48) hours of yesterday

3. Calendar features: - Day-of-week indicator variables (6 dummies for 7 days) - Month indicator variables (11 dummies for 12 months) - Public holiday indicator

4. Exogenous features (where available): - Demand forecast for hour h - Wind speed forecast - Solar CSI forecast - Temperature deviation

Per-Hour Models: Why Separate Is Better

The per-hour architecture is LEAR's most distinctive design choice. Instead of one model with 48 outputs, LEAR builds 48 independent models — each one a specialist for its particular time of day.

> **Per-hour models (per-period models):** An approach where a separate forecasting model is trained for each time period of the day. Each model sees only the data relevant to its specific period and learns hour-specific patterns. This contrasts with a "pooled" approach that trains a single model on data from all hours, using hour indicators to differentiate.

Advantages of per-hour models:

Natural hour-specific patterns. The 1pm model can learn that solar CSI is critical for midday prices, while the 3am model ignores CSI entirely (it is always zero at night). No interaction terms needed.

Feature selection is hour-specific. LASSO can select different features for each hour. The 6pm model might use lag-48 price, demand, and wind (the evening ramp drivers), while the 2am model might use only lag-48 price and a weekday indicator.

Simplicity. Each individual model is small (a few dozen features, one output) and trains in milliseconds. The computational cost is trivial even for daily refitting.

Interpretability. You can inspect each model's non-zero coefficients and verify they make physical sense: "Does the 1pm model use solar features? Yes. Does the 3am model? No. Good."

Disadvantages of per-hour models:

No information sharing. If lag-48 is important at hour 14, this does not help hour 15's model learn the same pattern. Each model must discover it independently from its own training data.

Prediction discontinuities. Adjacent-hour models are estimated independently, so their predictions can jump: hour 14's model predicts $80, hour 15's model predicts $120, creating an implausible $40 jump. In practice, these jumps are usually small because the underlying data is smooth.

More total parameters. 48 models × (say) 10 non-zero coefficients each = 480 effective parameters, versus perhaps 100 for a pooled model with hour interactions.

> **Specialisation wins:** Despite the disadvantages, the per-hour approach is remarkably robust in practice. The hourly structure of electricity prices is so strong that the benefit of specialisation (each model learns its hour's specific dynamics) outweighs the cost of isolation (no information sharing). Lago et al. (2021) showed that LEAR with per-hour models outperforms pooled alternatives across multiple markets and time periods.

> **Real-world example — what LEAR's models look like:** After fitting LEAR to SA1 data, the 1pm model might have non-zero coefficients on: yesterday's 1pm price (β = 0.45), last week's 1pm price (β = 0.20), today's solar CSI forecast (β = –15.3, negative because high solar reduces price), demand forecast (β = 0.08), and a January indicator (β = 12.5, capturing summer price premiums). All other features — including yesterday's

3am price, last night's wind, and the Tuesday indicator — have coefficients of exactly zero. LASSO has automatically determined that these features are not useful for predicting the 1pm price, given the other features already in the model.

Feature Selection via the L1 Penalty

How LASSO Selects Features

LASSO does not require the user to manually choose which features to include. Instead, the modeller provides a **deliberately large** candidate feature set, and LASSO's L1 penalty automatically selects the relevant subset:

1. Start with all candidate features (50+ features per hour).
2. The L1 penalty shrinks coefficients toward zero.
3. At the optimal λ (selected by LassoCV), many coefficients are *exactly* zero.
4. The features with non-zero coefficients are the "selected" features — the ones LASSO has determined are genuinely useful for prediction.

This automatic feature selection is enormously valuable because:

- It removes the need for manual feature engineering trial-and-error
- It adapts to different hours (selecting different features for each)
- It adapts over time (when the model is refitted, the selected features may change)
- It provides interpretability (you can see which features survived the selection process)

The Regularisation Path and Feature Importance

As λ decreases from large to small, features "enter" the model in order of importance:

Feature entry order: In the LASSO regularisation path, each feature has a critical λ value at which its coefficient first becomes non-zero. Features that enter at larger λ values are more important — they contribute enough predictive power to overcome a strong penalty. Features that enter only at very small λ values are marginal — they contribute little beyond what earlier features already capture.

For a typical hour in the NEM:

1. **First to enter (largest λ):** lag-48 price (yesterday's same-hour price). Always the most important feature.
2. **Second/third:** lag-336 price (last week) and/or demand forecast.
3. **Middle:** Cross-hour prices, temperature deviation, wind speed.
4. **Last to enter (smallest λ):** Month indicators, distant lags, weak calendar effects.
5. **Never enter (remain zero even at optimal λ):** Irrelevant features (solar at night, distant cross-hour prices with no physical connection).

This entry order provides a natural, data-driven **feature importance ranking** without any additional computation.

LASSO vs. Ridge for Electricity Prices

Ridge regression (L2 penalty) is the other common regularisation approach. For electricity prices, LASSO (L1) is preferred:

Property	LASSO (L1)	Ridge (L2)
Feature selection	Yes — sets irrelevant coefficients to exactly zero	No — all coefficients remain non-zero
Interpretability	High — you can see which features matter	Lower — all features contribute, hard to distinguish signal from noise
Performance with many irrelevant features	Good — irrelevant features are eliminated	Worse — irrelevant features add noise through small coefficients
Performance with correlated features	Can be unstable — may arbitrarily select one of two correlated features	More stable — spreads weight across correlated features
Computational cost	Slightly higher (iterative solver)	Lower (closed-form solution)

Elastic Net: A regularisation method that combines the L1 (LASSO) and L2 (Ridge) penalties: $\lambda_1 \times \Sigma|\beta_j| + \lambda_2 \times \Sigma\beta_j^2$. Elastic Net inherits LASSO's feature selection (from the L1 part) and Ridge's stability with correlated features (from the L2 part). It is sometimes used as a compromise when pure LASSO is unstable due to high feature correlation. In practice, for electricity prices, pure LASSO performs well because the key features (lag-48 price, demand) are not highly correlated with each other.

In practice, LEAR with LASSO typically outperforms Ridge by 5–15% in MAE on NEM data. The feature selection provided by L1 is particularly valuable because the LEAR feature set is deliberately large (many cross-hour lags, calendar indicators, weather variables), and most features are irrelevant for any given hour.

Time Series Fundamentals for LEAR

Stationarity

> **Stationarity:** A time series is stationary if its statistical properties — mean, variance, and autocorrelation structure — do not change over time. Stationarity is an assumption (explicit or implicit) of most statistical models, including linear regression. If the data-generating process changes (e.g., a new solar farm is built, shifting the midday price distribution), a model trained on old data becomes progressively less accurate.

Electricity prices are **not** stationary. They exhibit:

- **Trending mean:** Fuel costs change, carbon policies evolve, renewable penetration grows. The average price in 2020 was different from 2024.
- **Time-varying variance:** Summer months have more extreme prices (volatility) than spring and autumn.
- **Changing autocorrelation:** As the generation mix evolves (more solar, less coal), the relationship between hours changes — the midday solar dip deepens year over year.

LEAR handles non-stationarity through three mechanisms:

1. **The arcsinh transform** stabilises the variance (a form of variance-stabilising transform).
2. **Rolling window retraining** ensures the model sees only recent, relevant data.
3. **The per-hour structure** allows each hour to adapt independently to changing conditions.

The Training Window Length

Choosing the right training window length is a bias-variance tradeoff with a temporal dimension:

- **Too short (e.g., 3 months):** Low bias (adapts quickly to changes) but high variance (too few observations for stable coefficient estimation). May miss seasonal patterns — a 3-month summer window does not see winter dynamics.
- **Too long (e.g., 5+ years):** Low variance (many observations) but high bias (old data from a different market structure misleads the model).
- **Sweet spot (2–4 years for NEM):** Enough data to capture seasonal patterns and estimate stable coefficients, but recent enough that the market structure has not changed dramatically.

> **Real-world example — window length matters:** For SA1 in 2024, a 2-year training window (2022–2024) captures the current market structure (high solar penetration, significant battery fleet, post-coal-closure dynamics). A 6-year window (2018–2024) includes data from when solar penetration was half its current level and several coal plants were still operating. The 2018 data teaches the model relationships that no longer hold — for example, that midday prices are moderate (set by coal), when in 2024 they are often negative (set by solar surplus).

Forecast Horizons and Error Growth

Forecast error increases with the horizon, but the pattern depends on the model type:

- **Steps 1–6 (0.5–3 hours ahead):** Autoregressive momentum dominates. The most recent price is highly informative, and errors are small.
- **Steps 12–24 (6–12 hours ahead):** The diurnal pattern dominates. Calendar features carry most of the predictive power. Autoregressive features become less relevant.
- **Steps 24–48 (12–24 hours ahead):** Both autoregressive and calendar signals weaken. Weather features become increasingly important for predicting tomorrow's supply–demand balance.

LEAR's per-hour architecture sidesteps the horizon problem. Each model always predicts at the same effective horizon relative to its daily cycle — the "1pm model" always predicts one specific time of day, not a range of horizons. There is no explicit horizon management.

Model Evaluation and Comparison

Evaluating LEAR's Performance

LEAR is evaluated using the rolling-origin backtest framework from Chapter 5. Key metrics:

- **MAE** in $/MWh (absolute accuracy)
- **rMAE** relative to the similar-day naive (relative improvement)
- **Capture ratio** from LP dispatch (economic value)
- **Diebold-Mariano p-value** against the baseline (statistical significance)

A well-tuned LEAR model for SA1 typically achieves:

Metric	Typical LEAR value	Similar-day naive	Improvement
MAE	$15–20/MWh	$20–28/MWh	20–30%

rMAE	0.70–0.85	1.00 (by definition)	15–30%
Capture ratio	0.55–0.70	0.35–0.50	10–25 pp

These numbers vary with the test period, region, and exact feature set, but the pattern is consistent: LEAR substantially outperforms the naive baseline.

The Diebold-Mariano Test for Model Comparison

When comparing LEAR to the naive baseline (or to other models), the Diebold-Mariano test determines whether the MAE difference is statistically significant:

DM statistic = mean(d_t) / σ*(d_t)

where d_t = |e_naive,t| – |e_LEAR,t| is the loss differential at each time step, and σ* is the HAC-corrected standard error.

> **Loss differential:** The difference in forecast error between two models at each time step: $d_t = L(e_1,t) – L(e_2,t)$, where L is the loss function (absolute error, squared error, etc.). A positive d_t means model 1 had a larger error than model 2 at time t. The Diebold-Mariano test assesses whether the average loss differential is significantly different from zero.

If the DM p-value is below 0.05, we conclude that LEAR's improvement over the baseline is statistically significant — not just a lucky result on this particular test period. This discipline is essential in electricity price forecasting, where short test sets (a few months) can produce misleading comparisons.

> **The full evaluation pipeline:** Every model in this course is evaluated using the same rigorous pipeline: (1) rolling-origin backtest with embargo, (2) MAE and rMAE in dollar space, (3) capture ratio from LP dispatch, (4) Diebold-Mariano test against the prior best model. This standardised evaluation prevents cherry-picking results and ensures genuine progress.

Recalibration and Concept Drift

Why Models Degrade Over Time

A trained model is a snapshot of the statistical relationships that held during the training period. As the market evolves, these relationships change — a phenomenon called **concept drift**.

> **Concept drift:** A change over time in the statistical relationships between features and the target variable. In electricity markets, concept drift occurs when the underlying market structure changes — new generators are built, old ones retire, fuel prices shift, regulations change, demand patterns evolve. A model trained on old data captures old relationships that may no longer hold, leading to degraded performance.

Sources of concept drift in the NEM:

- **New generation capacity:** A new 200 MW wind farm changes the supply stack, shifting the net-load-to-price relationship. The model's learned coefficients on wind features become stale.
- **Generator retirement:** A coal plant closing removes baseload supply, structurally raising prices during periods that were previously cheap.
- **Fuel price changes:** A doubling of gas prices shifts the marginal cost of gas generators, changing the hockey-stick curve's shape and threshold.
- **Demand trends:** Growing electric vehicle charging shifts the demand profile. Increasing rooftop solar reduces daytime grid demand.
- **Regulatory changes:** Changes to the market price cap, bidding rules, or interconnector ratings alter market dynamics.

Refit Frequency

LEAR should be refitted regularly — typically **daily** (retrain using all data up to yesterday) or **weekly**:

Refit frequency	Pros	Cons
Daily	Fastest adaptation to changes	Highest computational cost (trivial for LEAR, significant for ML models)
Weekly	Good balance of adaptation and cost	Slight delay in responding to sudden changes
Monthly	Lowest cost	May miss important market shifts
Never	Zero cost	Guaranteed degradation over time

For LEAR, daily refitting is computationally trivial — fitting 48 LASSO regressions on 2 years of daily data takes seconds. There is no reason not to refit daily.

Monitoring for Degradation

Track the rolling rMAE (LEAR's MAE divided by the naive's MAE) over time. A well-functioning model should maintain rMAE below 1.0 consistently. Warning signs:

- **rMAE > 1.0 for a single week:** Possibly normal variation. Monitor but do not intervene.
- **rMAE > 1.0 for multiple consecutive weeks:** The model is underperforming the naive. Investigate — has the market changed? Are features stale? Is the training window too long or too short?
- **Sustained rMAE increase (upward trend):** Concept drift is degrading performance. Consider adjusting the training window, adding new features, or switching to a more adaptive model.

> **Real-world example — concept drift in action:** In late 2022, Australian gas prices spiked dramatically due to global energy market disruptions. The cost of gas-fired electricity generation doubled overnight. A LEAR model trained on 2020–2022 data (mostly low gas prices) had learned that net load of 2,000 MW corresponds to prices of ~$100 (gas peakers at $100/MWh marginal cost). After the gas price spike, the same net load produced prices of ~$200. The model's rMAE jumped from 0.75 to 1.1 until it accumulated enough post-spike data to recalibrate. Daily refitting resolved the problem within 2–3 weeks as the new gas-price regime entered the training window.

Glossary

Term	Definition
LEAR	LASSO Estimated AutoRegressive — per-hour LASSO models for price forecasting
Linear regression	Model predicting target as weighted sum of features
OLS	Ordinary Least Squares — standard coefficient estimation method
Coefficient	Weight on a feature in a linear model
Intercept	Constant term in a linear model
Residual	Difference between actual and predicted value
Overfitting	Model learns noise, performs well on training data but poorly on new data
Bias	Error from overly simplistic model assumptions
Variance	Error from model sensitivity to specific training data
Regularisation	Adding a penalty to control model complexity
Ridge (L2)	Regularisation with squared-coefficient penalty; shrinks but keeps all features
LASSO (L1)	Regularisation with absolute-coefficient penalty; drives some coefficients to zero
L1 norm	Sum of absolute values of coefficients
Sparsity	Property of having most coefficients exactly zero
Elastic Net	Combination of L1 and L2 penalties
Hyperparameter	Setting that controls the learning algorithm (e.g., λ)
Cross-validation	Estimating generalisation error by training/validating on data splits
LassoCV	LASSO with automatic λ selection via cross-validation
Regularisation path	Sequence of LASSO solutions as λ varies
Time-series CV	Cross-validation respecting temporal ordering
Per-hour models	Separate model for each hour of the day
Stationarity	Statistical properties constant over time
Concept drift	Change in feature–target relationships over time
Feature selection	Identifying which features are useful; automated by LASSO
Feature entry order	Order in which features gain non-zero coefficients as λ decreases
Loss differential	Difference in forecast error between two models at each time step
Diebold-Mariano test	Statistical test for significance of forecast accuracy differences

Summary

LEAR (LASSO Estimated AutoRegressive) is the standard benchmark for day-ahead electricity price forecasting, introduced by Lago et al. (2021). It fits separate LASSO regressions for each hour (or half-hour) of the day, allowing hour-specific feature selection and coefficient

estimation. LASSO's L1 penalty drives irrelevant feature coefficients to exactly zero — performing automatic feature selection from a deliberately large candidate set — while retaining only the features that genuinely improve prediction for each specific hour. The geometric intuition is that the diamond-shaped L1 constraint region has corners on the coordinate axes where coefficients are zero, while Ridge's circular L2 constraint does not. LassoCV selects the regularisation strength automatically via time-series cross-validation, making LEAR effectively parameter-free. The per-hour architecture naturally captures the strong hourly variation in price dynamics — solar features dominate midday models, demand features dominate evening models, and nighttime models rely primarily on autoregressive lags. Model evaluation follows the rigorous pipeline established in Chapter 5: rolling-origin backtest with embargo, MAE and rMAE in dollar space, capture ratio from LP dispatch, and Diebold-Mariano testing for statistical significance. Regular refitting (ideally daily) combats concept drift as fuel prices, the generation mix, and demand patterns evolve over time.